Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026

Plan du cours

CHAPITRE 7 : POO PYTHON

1. Introduction
2. Classes
3. Attributs
4. Méthodes
5. Héritage
6. Gestion des exceptions

I. Introduction

- Python a été initialement conçu comme un langage procédural, avec une forte inspiration des scripts.
- La Programmation Orientée Objet (POO) est pleinement supportée en Python et largement utilisée dans des projets professionnels.
- Comparée à Java, la POO en Python peut sembler moins “stricte”, car le langage offre plus de flexibilité.
- Java impose une structure orientée objet plus rigoureuse, ce qui le rend souvent plus formel pour la conception de classes.

Remarque :

Python et Java sont tous deux orientés objet, mais leur philosophie diffère :

- Python privilégie la simplicité et la lisibilité,
- Java privilégie la structure et la robustesse.

II. Classes

Déclaration d'une classe en Python

```
class NomClasse:
    # déclarations (attributs et méthodes)
    pass
```

Remarque :

- Le nom de la classe suit généralement la convention **CamelCase**.
- Le mot-clé **pass** permet de définir **une classe vide.**

II. Classes

Le constructeur en Python : __init__

- Définition :

```
def __init__(self, parametre1, parametre2, ...):
    # initialisation de l'objet
```

- Points clés :
  - **__init__** est une méthode spéciale (méthode magique).
  - Elle est **appelée automatiquement** lors de la **création d'un objet**.
  - Elle sert à **initialiser les attributs de l'instance**.

II. Classes

Exemple : Classe Point

```
class Point:
        def __init__(self, x, y):
                self.x = x
                self.y = y
# Programme principal
p1 = Point(2, 5)
```

Remarques :

- x et y sont des **attributs d'instance**.
- **Chaque objet Point possède ses propres valeurs de x et y.**

III. Attributs

Attributs de classe vs Attributs d'instance

En Python, les attributs peuvent être :

1. Attributs d'instance :

Déclarés dans le constructeur __init__ :

```
class Point:
        def __init__(self, x, y):
                self.x = x
                self.y = y
```

- Propres à chaque objet
- Dépendent de l'instance créée

III. Attributs

Attributs de classe vs Attributs d'instance

En Python, les attributs peuvent être :

2. Attributs de classe

Déclarés directement dans la classe :

```
class Point:
        dimension = 2 # attribut de classe
        def __init__(self, x, y):
                self.x = x
                self.y = y
```

- Partagés par toutes les instances
- Accessibles via **Point.dimension** ou **via un objet**

III. Attributs

Encapsulation en Python

- Python ne propose pas de véritable encapsulation stricte comme en Java (private, protected).
- Il repose sur une **convention de confiance** entre le développeur et l'utilisateur de la classe. Il fait confiance au programmeur: "*Nous sommes tous adultes ici*".

Conventions utilisées :

_attribut : usage interne (convention)

__attribut : name mangling (pseudo-privé)

```
class Point:
        def __init__(self, x, y):
                self.__x = x # pseudo-privé
                self._y = y # usage interne
```

IV. Méthodes

Types de méthodes en Python

En Python, une classe peut contenir trois types de méthodes :

- **méthodes d'instance**
- **méthodes de classe**
- **méthodes statiques**

IV. Méthodes

1. Méthodes d'instance

Syntaxe générale :

```
def nom_methode(self, parametre1, parametre2, ...):
        # déclarations
```

Remarques :

- **self** doit être le premier paramètre de toute méthode d'instance.
- **self** représente une référence à l'objet courant → équivalent de **this** en Java.
- Les attributs de l'objet doivent être accédés via la référence **self**.
- Oublier **self** est une erreur fréquente chez les débutants.

IV. Méthodes

1. Méthodes d'instance

- Exemple : Méthode d'instance "*translate*" dans la classe Point

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def translate(self, dx, dy):
        self.x += dx
        self.y += dy
```

- Utilisation :

```
p = Point(2, 5) # Création de l'objet p
p.translate(1, -2) # objet.méthode(paramètres)
Point.translate(p, 1, -2) # ou Class.méthode(objet, paramètres)
```

IV. Méthodes

2. Méthodes de classe (@classmethod)

```
class Point:
    compteur = 0
    def __init__(self, x, y):
        self.x = x
        self.y = y
        Point.compteur += 1
    @classmethod
    def nombre_points(cls):
        return cls.compteur
```

IV. Méthodes

2. Méthodes de classe (@classmethod)

- Caractéristiques :
  - Décorateur **@classmethod**
  - Premier paramètre : **cls** (la classe)
  - Accès aux **attributs de classe**
  - Appelées sur la **classe** ou **sur un objet**
  - print(Point.nombre_points())
- Usage typique :
  - Méthodes liées à la classe
  - Compteurs, méthodes de création alternatives (factory methods)

IV. Méthodes

3. Méthodes statiques (@staticmethod)

- Une méthode statique est une méthode définie à l'intérieur d'une classe, qui ne dépend ni de l'objet (self) ni de la classe (cls), et qui se comporte comme une fonction utilitaire liée logiquement à la classe.

```
class Point:
    @staticmethod
    def distance(p1, p2):
        return ((p1.x - p2.x)**2 + (p1.y - p2.y)**2) ** 0.5

d = Point.distance(p1, p2)
```

- Caractéristiques :
  - Décorateur **@staticmethod**
  - Aucun paramètre implicite (self ou cls)
  - Fonction utilitaire liée logiquement à la classe

IV. Méthodes

Surcharge de méthodes en Python

- En Python : pas de surcharge classique

```
class Test:
    def f(self, x):
        print(x)
    def f(self, x, y):
        print(x, y)
```

- Résultat :
  - La deuxième (***f(self, x, y)***) méthode écrase la première (***f(self, x)***)
  - Python ne supporte pas la surcharge par nombre ou type de paramètres

IV. Méthodes

Surcharge de méthodes en Python

Comment simuler la surcharge en Python ?

- paramètres par défaut

```
def afficher(self, x, y=None):
    if y is None:
        print(x)
    else:
        print(x, y)
```

- **x** : Paramètre obligatoire
- **y=None** :Paramètre optionnel avec une valeur par défaut
- Cela signifie que la méthode peut être appelée avec un seul argument ou avec deux arguments
- Python ne supporte pas la surcharge de méthodes comme Java. Cette écriture permet de simuler plusieurs versions de la méthode.

IV. Méthodes

Surcharge d'opérateurs en Python

- La surcharge d'opérateur permet de redéfinir le comportement des opérateurs intégrés (+, -, *, ==, etc.) pour les objets d'une classe.
- Python appelle automatiquement les méthodes spéciales correspondantes à l'opérateur.

Opérateur	Méthode spéciale	Exemple
+	__add__(self, autre)	additionner deux objets
-	__sub__(self, autre)	soustraction
*	__mul__(self, autre)	multiplication
==	__eq__(self, autre)	égalité
<	__lt__(self, autre)	comparaison "moins que"
>	__gt__(self, autre)	comparaison "plus que"
str()	__str__(self)	conversion en chaîne
len()	__len__(self)	longueur

IV. Méthodes

Surcharge d'opérateurs en Python

- Exemple : Classe Point et opérateur +

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, autre):
        return Point(self.x + autre.x, self.y + autre.y)
    def __str__(self):
        return f"({self.x}, {self.y})"
```

- Utilisation

```
p1 = Point(2, 3)
p2 = Point(5, 7)
p3 = p1 + p2 # appelle automatiquement p1.__add__(p2) qui  retourne un nouvel objet Point
print(p3) # Affiche : (7, 10)
```

V. Héritage

1. Héritage simple

- Permet de créer une nouvelle classe basée sur une classe existante.
- La nouvelle classe hérite des attributs et méthodes de la superclasse.

Syntaxe :

```
class NomClasseFille(NomSuperClasse):
    # déclarations
```

- Exemple

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
class Point3D(Point): # Point3D étend Point
    def __init__(self, x, y, z):
        super().__init__(x, y) # appelle le constructeur de Point
        self.z = z
```

V. Héritage

2. Héritage multiple

- Python permet de dériver une classe de plusieurs superclasses.

Syntaxe :

class NomClasseFille(SuperClasse1, SuperClasse2, ...):

 # déclarations

- Règle de résolution des conflits :
  - Si plusieurs superclasses ont le même attribut ou méthode, Python utilise l'ordre de gauche à droite.
  - Cela fait partie de la MRO (Method Resolution Order).

V. Héritage

2. Héritage multiple

- Exemple d'héritage multiple

```
class A:
        def afficher(self):
                print("Classe A")

class B:
        def afficher(self):
                print("Classe B")

class C(A, B):
        pass

c = C()
c.afficher() # Affiche : "Classe A" (A est à gauche)
```

V. Héritage

Appel du constructeur de la superclasse

- Option 1 : Référence directe à la classe

```
class Point3D(Point):
    def __init__(self, x, y, z):
        Point.__init__(self, x, y) # appel du constructeur de Point
        self.z = z
```

- Option 2 : Utilisation de **super()** (recommandée)

```
class Point3D(Point):
    def __init__(self, x, y, z):
        super().__init__(x, y) # plus propre et flexible
        self.z = z
```

Remarques :

- super() fonctionne également avec héritage multiple (Method Resolution Order).
- Permet d'éviter de référencer directement le nom de la superclasse.

V. Héritage

Appel d'une méthode de la superclasse

- Exemple : méthode "*translate*" pour Point3D

```
class Point3D(Point):
        def translate(self, dx, dy, dz):
                Point.translate(self, dx, dy) # appel direct à la méthode de Point
                # ou super().translate(dx, dy) # version recommandée
                self.z += dz
```

- On réutilise le code de la superclasse pour mettre à jour x et y.
- La classe fille peut ajouter du comportement spécifique (z ici).

❖ **Utilisation**

```
p = Point3D(1, 2, 3)
        p.translate(2, -1, 5)
        print(p.x, p.y, p.z) # Affiche : 3, 1, 8
```

VI. Gestion des exceptions

Structure générale

```
try:
        # Code à risque (susceptible de provoquer une erreur)
except ExceptionType:
        # Code exécuté si l'exception se produit
```

Principe :

- Le code dans le bloc try est exécuté normalement.
- Si une exception survient, l'exécution du try s'arrête.
- Python cherche un bloc except correspondant au type de l'erreur.

Exemple : Division par zéro

```
try:
        x = 10 / 0
except ZeroDivisionError:
        print("Division par zéro interdite !")
```

VI. Gestion des exceptions

Plusieurs exceptions possibles

```
try:
        x = int("abc")
except ValueError:
        print("Conversion impossible")
except ZeroDivisionError:
        print("Division par zéro")
```

- Python teste les except dans l'ordre.

VI. Gestion des exceptions

Exception générique

```
try:
        x = int("abc")
except Exception as e:
        print("Erreur :", e)
```

- Attraper ***Exception*** est pratique pour le debug
- Mais à éviter dans le code final si possible

VI. Gestion des exceptions

Structure complète des exceptions en Python

```
try:
        # Code à risque
except:
        # Code exécuté en cas d'erreur
else:
        # Code exécuté si AUCUNE exception n'a été levée
finally:
        # Code toujours exécuté
```

VI. Gestion des exceptions

Lever une exception en Python (**raise**)

raise ExceptionType("message")

Principe :

- Permet de signaler explicitement une erreur
- Interrompt l'exécution normale du programme
- Transmet l'erreur à l'appelant

Types d'exceptions courantes :

Type	Signification
ArithmeticError	Erreurs arithmétiques
AssertionError	Erreur d'assertion
IndexError	Indice invalide
NameError	Nom non défini
SyntaxError	Erreur de syntaxe
TypeError	Type incorrect
ValueError	Valeur invalide

VI. Gestion des exceptions

Exemple (raise) : Classe BankAccount

```
class BankAccount:
        def __init__(self, balance=0):
                self.balance = balance
        def deposit(self, amount):
                if amount < 0:
                        raise ValueError("negative amount")
                self.balance += amount
```

Utilisation :

```
account = BankAccount(100)
try:
        account.deposit(-50)
except ValueError as e:
         print("Erreur :", e)
```

VI. Gestion des exceptions

Définition d'une exception personnalisée

- Python permet de créer ses propres types d'exceptions
- Une exception personnalisée hérite de Exception

Syntaxe :

```
class InvalidPointError(Exception):
    pass
```

- Cette exception ne fait rien de plus que Exception, mais elle possède un nom explicite et significatif.

Lever une exception personnalisée

```
raise InvalidPointError("Point invalide")
```

- Le message permet de décrire précisément l'erreur.

Merci de votre attention

Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026